Sundarban Security Self-Assessment

Sundarban: Security Self-Assessment

FinTechathon 2026 · International Track · Topic F (Agent Wallet) · issue #19 Repository: https://github.com/Bijoy2406/Sundarban · Assessed on 18 Oct 2026, branch `bijoy`

This is the team's own assessment of what Sundarban protects against, how, and what it does **not** protect against. Every rule, threshold and file reference below was read from the code on the date above. Every number was measured by running the command shown next to it. Where something could not be checked it says *(not verified)*.

One-sentence summary. The language model only turns text into a proposed action; a deterministic Python firewall (no AI) decides *allow / ask / deny*; a signed, single-use approval token tied to the exact calldata is required to send; and every decision is written to a hash-chained audit log.

Headline	Result	How to reproduce
Red-team cases that behaved as expected	**55/55** (40 attacks, 15 legitimate)	`python -m firewall.redteam.run --no-write`
Attacks denied / held for a human / let through	36 / 4 / **0** of 40	same
Legitimate requests wrongly denied	**0/15** (5/15 needed a confirmation)	same
Same attacks through both real agents (local Anvil)	protected agent executed **0/40**; naive agent executed **38/40**	`python -m firewall.redteam.run --agents --no-write`
Automated tests	**1235 passed, 1 skipped** with Anvil; 1173 passed, 63 skipped without	`python -m pytest -q`
Real Sepolia transactions through the full pipeline	**6 sent, 2 denied** (table in section 7.3; the wallet's complete history of 15 sends is in 7.4)	`evidence/tx_hashes.csv`
Audit-log head anchored on Sepolia	verified `RESULT: OK`	`ACTIVE_NETWORK=sepolia python wallet_engine/scripts/anchor_audit.py --verify`

1. Scope and assumptions

In scope	The wallet engine (`wallet_engine/`), the firewall (`firewall/`), the agents and replay (`agent/`), the HTTP API (`api/`), the UI (`ui/`), the audit log and its Sepolia anchor, the red-team data and runner.
Money	**Test tokens only** (TestUSD, "TUSD"). Local Anvil (chain 31337) and Ethereum Sepolia (11155111). No mainnet, no real funds, anywhere.
Deployment model	A single-user demo on one computer: one backend process holds the firewall, the approval secret, the agent and the signing key. Limits are demo-sized.
Threats considered	(a) prompt injection through public posts, NFT metadata, tool output and chat; (b) a fooled or buggy model proposing a harmful action; (c) confused-deputy and look-alike-address tricks; (d) tampering with the evidence (log edits, truncation); (e) leaking a secret through an error message.
Not considered	A malicious operator with shell access to the machine, a compromised dependency, a compromised RPC provider, real-money regulatory compliance (KYC, sanctions, travel rule), and attacks on the Solidity token (it is a plain ERC-20 written for the demo, not audited).

2. How a request is decided

```
text ─► LLM (intent only: action, token, to, amount, unlimited, rationale)      agent/intents.py
        origin ("user_chat", "public_feed", ...) is set by CODE from the channel  api/main.py, agent/fee
d.py
          │
          ▼
      wallet_engine.plan  ─►  TxPlan  (the exact calldata that would be signed)    wallet_engine/erc20.py
      wallet_engine.simulate ► SimResult (eth_call + estimate_gas)                 wallet_engine/simulat
e.py
          │
          ▼
      firewall.evaluate  ─►  allow | ask | deny      (+ rule ids, plain-English reasons)   firewall/polic
y.py
          │ allow: one-use HMAC token bound to the calldata
          │ ask:   a person confirms outside the chat ─► firewall.confirm re-checks and issues the token
          ▼
      wallet_engine.execute  − re-verifies the token against the calldata, re-simulates, signs, sends
          ▼
      tx hash + audit log entry (hash-chained)
```

---

3. Permission tiers

3.1 The four tiers

Tier	What it covers	Who decides	Enforced by
0. Read	Balances, allowances, Preview, audit-log view, feed list. Nothing is signed, counted or remembered.	Always available	`wallet_engine.balances/approvals`; `firewall.preview` (no side effects); `GET /api/*`
1. Small, allowlisted (auto)	A transfer or limited approval **up to 50 TUSD** to an **allowlisted** address, requested from `user_chat`, with no rule firing. A revoke (taking a permission away) has no amount rules.	The firewall alone	`policy.evaluate` returns `allow` and a token
2. Confirmed (ask)	New recipient, amount above 50 TUSD (up to 500), or more than 5 actions in 10 minutes. Nothing is sent until a person confirms.	Firewall asks; a person answers; the firewall re-checks	`policy.evaluate` returns `ask`; `policy.confirm`
3. Denied	Everything in the deny rows below. No confirmation can override a deny.	The firewall alone	`policy.evaluate` / `policy.confirm` return `deny`

Precedence: any deny wins; otherwise any ask; otherwise allow. **The default is deny** (rule DEF-01): a request that cannot be understood, or any internal error, ends in deny and never in allow (`policy._safe_assess`, `policy.evaluate`).

3.2 Every rule, with its exact threshold and enforcement point

Thresholds are read from `firewall/rules.yaml` (checked in); all enforcement is in `firewall/policy.py` unless another file is named.

Rule	Tier	Triggers when	Threshold / setting	Enforced in
PRV-01	deny	the request's `origin` is not a trusted channel	`trusted_origins: [user_chat]`; exact string match	`provenance.check`, called from `_assess`
KIL-01	deny	the kill switch is on	flag file `logs/kill_switch.flag` + memory; any value except `False` turns it on	`_assess`; `state.kill_switch_on`; `tokens.verify_token` also refuses every token while it is on

Rule	Tier	Triggers when	Threshold / setting	Enforced in
NET-01	deny	the plan's chain id differs from the expected chain	`chain_id` in rules.yaml; `null` **today**, so the expected chain is read from the node (see risk R-01)	`_assess`, `_expected_chain_id`
TOK-01	deny	the token contract is not ours	`TOKEN_ADDRESS` in `.env` must match both the request and the calldata target	`_assess`, `_allowed_tokens`
APR-01	deny	an approval for 2^255 or more (unlimited)	`UNLIMITED_FROM = 2**255`	`_assess`
APR-02	deny	an approval to a spender not on the allowlist	allowlist (2 addresses: Alice, Bob)	`_assess`
ADR-01	deny	an address not on the allowlist that shares 6 or more leading + trailing hex digits with one that is	`LOOKALIKE_CHARS = 6`	`_looks_like`, called from `_assess`
DST-01	deny	a transfer to the zero address, `0x…dEaD`, or the token contract itself	fixed set	`_assess`
SIM-01	deny	the calldata is not plain hex or not a supported call; **or** it does not match the request to the last digit (action, token, recipient, amount, unlimited flag, native value ≠ 0); **or** the dry run failed; **or** the dry run shows more outflow than requested (any outflow for approve/revoke)	n/a	`_assess` (the dry run itself is `wallet_engine/simulate.py:run`; `execute` re-simulates before signing)
LIM-01	deny	amount above the per-transaction maximum (transfers and approvals)	`max_tx: "500"` TUSD	`_check_amount`
LIM-03	deny	the rolling 24-hour total (transfers **and** approvals) plus this amount exceeds the cap	`daily_cap: "1000"` TUSD	`_check_amount`; `state.spent_last_24h`
CNF-01	deny	`confirm()` for a request nobody was asked about, already answered, expired, or whose transaction differs from the one asked	question stays open 600 s (`CONFIRM_WINDOW_SECONDS`)	`policy.confirm`
DEF-01	deny	anything unexpected: wrong types, unreadable plan or simulation, an internal exception, a token that cannot be issued, an audit entry that cannot be written, a spend that cannot be recorded	n/a	`_snapshot_*`, `_safe_assess`, `_finish`, `evaluate`
ALW-01	ask	a transfer to a recipient not on the allowlist	allowlist	`_assess`
LIM-02	ask	an amount above the auto limit (and not above the maximum)	`auto_limit: "50"` TUSD	`_check_amount`
VEL-01	ask	5 or more actions from trusted origins already happened in the window (the 6th and later ask)	`velocity: max_actions 5, window_seconds 600`	`_assess`; `state.actions_in_window`

How the engine stays honest about its own inputs (`firewall/policy.py`):

- **Snapshot first.** The request, plan and simulation are copied once with strict type checks; every rule then sees the same values, so an object that changes between reads cannot fool two different rules.
- **Numbers come from the calldata**, not from the request text, via `wallet_engine/binding.py:token_binding`, and must equal the request to the last digit (SIM-01).
- **No audit entry, no money.** An `allow` is only returned after the audit write succeeded; spending is recorded after that. If either fails the answer becomes deny (DEF-01).
- **Outside text is cleaned** (`_clean`: printable characters, shortened) before it is put into a reason or log entry.
- **Only trusted origins count towards VEL-01**, so an outsider posting many requests cannot throttle the owner.
- **Approvals count as spending** for the daily cap, because an allowance can be pulled out later.
- **`firewall.preview` has no side effects** (no velocity count, no spend, no token, no open question, no audit write). The Preview button uses it, so previewing cannot change a later decision. `tests/test_preview_redteam.py` (71 tests) checks that preview leaves no trace and agrees with evaluate on the 53 red-team cases that contain an action (the 2 read-only questions create no request).

---

4. Trust boundaries: where "origin" comes from

PRV-01 is only as good as the code that sets `origin`. It is set in trusted code, from the channel, never from text:

Channel	Origin value	Set by
The owner's chat box and the Preview / Execute form	`user_chat`	`api/main.py` (`chat` passes the literal; `_request` sets it)
A synthetic public post	`public_feed`	`agent/feed.py:FeedItem.origin`, derived from the item's `source` field
NFT metadata	`nft_metadata`	same
Attack replay	the feed item's origin	`agent/replay.py`; the browser sends only an attack id

- **The model cannot set it.** `agent/intents.py` accepts a reply only if it has exactly the six keys `action, token, to, amount, unlimited, rationale`; an extra key such as `origin` rejects the reply. `parse_intent` has no `origin` parameter.
- **The browser cannot set it.** `/api/execute` and `/api/plan` rebuild the plan on the server and ignore any decision sent by the client; `/api/confirm` takes only a request id.
- **Untrusted text is also fenced** in `<untrusted>` tags (with Unicode normalisation so full-width look-alikes of the closing tag are neutralised). This is defence in depth: the firewall does not depend on it.
- **Limit:** a message the owner types, or pastes, into the chat is `user_chat`, even if the owner copied it from an attacker. See risk R-12.

---

5. Controls beyond the rules

Control	What it does	Where	Evidence (tests)
Approval token	`"<expiry>.<HMAC-SHA256>"` over request id, action, token, recipient, amount and expiry; secret `APPROVAL_SECRET` (at least 16 characters); valid **120 s**; **single use** (the used-token memory is keyed by the signature and the expiry must be canonical digits, so `0…`, `+…` or `…` spellings of a spent token are refused); refused while the kill switch is on	`firewall/tokens.py`	`test_tokens.py` (22), `test_tokens_hardening.py` (7)
Calldata binding	`execute` re-derives action/token/recipient/amount from the bytes it will sign and verifies the token against them, so "approve A, send B" fails	`wallet_engine/binding.py`, `execute.py`	`test_simulate_execute.py` (27)
Execution guards	refuses unless the decision is `allow` with a valid token, the plan is for this wallet and chain, value is 0, gas is 21,000 to 500,000, and a fresh simulation passes; sends under a lock so two requests never share a nonce	`wallet_engine/execute.py`	`test_simulate_execute.py`, `test_execute_pending.py` (6)
Pending state	a transaction that was broadcast but has no receipt in 120 s is reported as *pending* with its hash, never as failed, so it is not sent twice. A send that raises after signing (timeout, dropped connection) is also reported as pending, with the hash computed from the signed transaction	`execute.py:ExecutionPending`, `agent/agent.py:is_pending`	`test_execute_pending.py`
Human confirmation	an `ask` is remembered on the server (600 s); only `/api/confirm` or the UI Confirm button continues it, after `firewall.confirm` re-runs the hard rules; chat text can never confirm. The agent's confirmation text shows the **full 42-character address**	`agent/agent.py`, `api/main.py`	`test_agent.py` (11), `test_api.py` (15), `test_e2e.py` (5), `test_live_agents_api.py` (13)
Spend ledger	the 24-hour total survives a restart; each line is HMAC-chained (key derived from `APPROVAL_SECRET`); an edited or half-written line makes the firewall refuse to act (verified: `StateCorruptError`)	`firewall/state.py`	`test_state.py` (294)

Control	What it does	Where	Evidence (tests)
Kill switch	one call freezes everything; any value other than the boolean `False` turns it on; an unreadable flag counts as on; the flag file survives a restart	`firewall/state.py`, `POST /api/kill-switch`	`test_state.py`, `test_tokens_hardening.py`
Address and amount checks on every path	the API forms apply the chat agent's full-address and EIP-55 checks (a wrong mixed-case checksum is a 400); amounts convert in a 200-digit Decimal context and a value that cannot be converted exactly is refused, not rounded; `evidence_run.py` refuses `--temp-allowlist` unless the chain is local and writes "human confirmation" only when the firewall allowed it	`api/main.py`, `agent/resolve.py`, `wallet_engine/erc20.py`, `evidence_run.py`	`test_api_address_check.py` (15), `test_erc20.py`, `test_evidence_run.py`
Approval token never leaves the server	the JSON serialiser blanks `approval_token` in every API response (execute, confirm, chat)	`api/main.py:_jsonable`	`test_api_token_hidden.py` (4)
Strict rules loader	unknown keys, duplicate keys, unquoted money values, a limit above the maximum or a zero-address allowlist entry all make `rules.yaml` unusable, and an unusable file denies everything	`firewall/state.py:load_rules`	`test_state.py`
Audit log	append-only JSON Lines; each line holds the SHA-256 of the previous one, so editing, deleting, inserting or re-ordering a line breaks `verify`; secret-looking keys are replaced by `[REDACTED]`; a damaged log is never silently extended; cross-process lock (`fcntl` on Linux/macOS, `msvcrt` on Windows, an exclusive-create lock file elsewhere)	`firewall/audit.py`, `scripts/verify_audit.py`	`test_audit.py` (29), `test_audit_locking.py` (8)
Sepolia anchor	the head hash of the submitted evidence log is written into a zero-value self-transaction (`sundarban-audit-v1:` + 32-byte head). Anyone can run `--verify` read-only. **It proves** the log existed in exactly this form at that block, and that anything anchored was not later rewritten or cut. **It does not prove** the entries are true, and it covers only the evidence log, only up to the anchored entry. It is a manual step	`wallet_engine/scripts/anchor_audit.py`	`test_anchor_audit.py` (27)
Secret redaction	URLs become `[url hidden]` and the values of `RPC_URL*`, `*_PRIVATE_KEY`, `*_API_KEY`, `*_SECRET` become `[hidden]` in anything an exception puts in front of a person; agent replies show only the exception *type*; the chat agent, every API error detail and the evidence script's CSV note and preflight message go through it	`shared/redact.py`; used in `simulate.py`, `chain.py`, `api/main.py`, `agent/agent.py`, `agent/replay.py`, `evidence_run.py`	`test_redact.py` (12)
Local-only demo parts	the drainer needs `ACTIVE_NETWORK=local` **and** chain id 31337; the replay needs both as well; the naive agent checks chain id 31337	`wallet_engine/attacks/drainer.py`, `agent/replay.py`, `agent/naive_agent.py`	`test_drainer.py` (17), `test_attack_replay.py` (23), `test_naive_agent.py` (3)
Separate keys per network	`WALLET_PRIVATE_KEY` is used only on local; `SEPOLIA_PRIVATE_KEY` only on Sepolia; the attacker key is a different Anvil account that is not on the allowlist	`wallet_engine/chain.py:key_var`	`test_drainer.py`

---

6. Mapping to the OWASP lists

Names and numbers were checked against the OWASP GenAI Security Project pages and OWASP's own `secure-agent-playbook` repository. The agentic list is *OWASP Top 10 for Agentic Applications for 2026* (published 9 Dec 2025); ASI02 is written "Tool Misuse" there and "Tool Misuse and Exploitation" in the full document. The LLM list is *OWASP Top 10 for LLM Applications 2025*.

6.1 LLM Applications 2025

ID and name	Threat in a wallet	Control in Sundarban	Evidence	Residual risk
LLM01 Prompt Injection	A public reply, an NFT description or tool output carries hidden or encoded instructions ("approve unlimited to 0x…BAD")	The model has no authority. Origin is set by code (PRV-01); the transaction itself is judged by APR-01, APR-02, ADR-01, LIM-*; untrusted text is fenced	Red-team: 11 attacks from outside channels, all denied by PRV-01; protected agent executed **0/40** attacks, naive agent **38/40**	The model can still be fooled into *proposing* something. Safe only because the firewall decides. 4/40 attacks are *held for a person*, not denied (R-13)
LLM02 Sensitive Information Disclosure	A key or an RPC API key appears in an error shown to the user or written to a log	`shared/redact.py`; the audit log redacts secret-looking keys; agent replies show only the exception type; keys are read from `.env` only	`test_redact.py`, `test_audit.py`	Message text itself goes to a third-party LLM relay (R-05). `.env` is plain text (R-04)
LLM05 Improper Output Handling	Model output used as a command	The reply must be JSON with exactly six keys, a plain-decimal amount, a known action; anything else is retried once and then refused. The result is *data* for the firewall, never executed	`test_intents.py` (33)	The model may produce a valid but harmful intent; see LLM01
LLM06 Excessive Agency	The agent can do more than the owner intends (amount, recipient, approval size)	The agent cannot sign: only `wallet_engine.execute` can, and only with a calldata-bound, single-use token. Limits: 50 auto, 500 max, 1000/day, 5 actions per 10 minutes; unlimited approvals and non-allowlisted spenders are denied; one token contract only (TOK-01)	`test_policy_preview.py` (267), red-team 55/55	Limits are demo values and the allowlist has two entries (R-13). A token minted inside the process bypasses the firewall (R-04)

6.2 Agentic Applications 2026

ID and name	Threat in a wallet	Control in Sundarban	Evidence	Residual risk
ASI01 Agent Goal Hijack	Text in a post or NFT changes what the agent is trying to do	The goal is not the agent's to change: allow/ask/deny comes from code. The replay shows the same hijacked action sent to both agents: the naive one pays, the protected one is blocked with reasons	`agent/replay.py`; `test_attack_replay.py`; `--agents` run: 0/40 vs 38/40	Owner pastes hostile text into the trusted chat (R-12)
ASI02 Tool Misuse	The agent calls the send tool with harmful parameters	The only send function needs an `allow` decision plus a token bound to the exact calldata; parameters are re-read from the bytes; re-simulation before signing	`test_simulate_execute.py`, `test_live_agents_api.py`	A process-level compromise can mint tokens (R-04)
ASI03 Identity & Privilege Abuse	Text claims "I am the admin, raise the limit"; a gifted NFT "grants" rights; a stronger identity is borrowed	There are no privileges in text. `trusted_origins` is a fixed list; limits and allowlist live in a file, not in chat; fake-admin and NFT cases are in the red-team set	red-team categories `fake_admin_claim`, `nft_metadata`, `public_feed` (all denied)	No login: every local caller of the API is "the owner" (R-03)
ASI06 Memory & Context Poisoning	Planted content changes later decisions	The agent keeps **no conversation memory or retrieval store**: each message is parsed alone with the fixed system prompt. The firewall's memory (spend ledger) is HMAC-chained and fails closed if damaged	`agent/intents.py:parse_intent` (messages = system prompt + this message only); `test_state.py`	Deleting the whole ledger file resets the daily cap; velocity memory and open questions live in process memory (R-02)
ASI09 Human-Agent Trust Exploitation	A fluent agent talks a person into approving a bad action	An `ask` is not decided in the chat; reasons are written by code from the actual transaction (not by the model); the confirmation text shows the full address; the calldata, contract and arguments appear on the Preview card	`test_agent.py`, `test_e2e.py`	A person can still click yes. 5/15 legitimate requests need a confirmation, so approval fatigue is real (R-13)

7. Evidence and how to reproduce it

7.1 Tests

Measurement	Result	Command
Full suite with local Anvil and TestUSD deployed	**1235 passed, 1 skipped** (about 33 s)	`anvil` (another terminal), `python wallet_engine/scripts/deploy.py`, `python -m pytest -q`
Full suite without Anvil	**1173 passed, 63 skipped** (the 63 need a chain and show as skipped, not passed)	`python -m pytest -q`
The one skip with Anvil	a retired check in `tests/test_evidence_run.py` that applied only to the old stub firewall	n/a
UI build	succeeds (Vite)	`cd ui && npx vite build`
Shared contract (types and signatures)	passes	`python -m pytest tests/test_contract.py -v`

Per-file test counts and what each file proves are in the README (section "Test cases").

7.2 Red-team suite (`firewall/redteam/`, 40 attacks in 14 categories, 15 legitimate cases)

Measure	Result
Cases behaving exactly as expected	**55/55**
Attacks denied outright	36/40 = 90.0%
Attacks held for a human (nothing sent)	4/40 = 10.0%
Attacks that went through on their own	**0/40**
Legitimate requests wrongly denied	**0/15 = 0.0%**
Legitimate requests that needed a confirmation	5/15 = 33.3%
Through both real agents on local Anvil	protected agent executed 0/40; naive agent executed 38/40 (27 transfers, 11 approvals; largest transfer 1,000,000 TUSD)

```
python -m firewall.redteam.run --no-write          # firewall only; needs no chain and no API key
python -m firewall.redteam.run --agents --no-write # also both agents; needs Anvil + deployed token; send
        s local test transactions only
```

The full per-case report is `evidence/redteam_report.md`. **What it does not show:** (a) it tests the firewall with structured requests, so it says nothing about how often a real language model is fooled (in `--agents` mode each case's fixed intent replaces the model); (b) SIM-01 has no case in the 55 and is proved by unit tests only; (c) the 2 kill-switch attacks reach the naive agent's firewall-free path but the engine refuses every token while the switch is on.

7.3 Sepolia transactions through the full pipeline (`evidence/tx_hashes.csv`)

Run on 16 Oct 2026 (UTC) with the real policy engine; the two `ask` rows were confirmed by a person at the prompt (the CSV note reads "typed at the prompt"). Wallet `0xB89C8A9826cac01E3b4A01873Bd0C27553567609`, token `0x81EBDCC80E45b57e774e13Ee4725963A872f0f47`.

#	Action	Firewall	Rules	Status	Transaction
1	transfer 10 TUSD to Alice	allow	ALLOW	sent	`0xd618054f…ee1eef`
2	transfer 30 TUSD to Bob	allow	ALLOW	sent	`0xc9e0d856…d5db74`
3	approve 20 TUSD for Alice	allow	ALLOW	sent	`0x852eeff0…08d7d0`
4	revoke Alice	allow	ALLOW	sent	`0xbc8af874…9c7000`

#	Action	Firewall	Rules	Status	Transaction
5	transfer 200 TUSD to Bob	**ask**, then confirmed	LIM-02	sent	`0x148f154e…47068f0bc`
6	transfer 1 TUSD to Alice	**ask**, then confirmed	VEL-01 (the 6th action in 10 minutes)	sent	`0x0ee5931f…7560dfc9`
7	unlimited approval for Alice	**deny**	APR-01, VEL-01	not sent	none
8	transfer 25 TUSD to `0x…0BAD` from a public post	**deny**	PRV-01, ALW-01, VEL-01	not sent	none

Rows 7 and 8 also list VEL-01 because every request counts towards the velocity limit. Other on-chain records: the TestUSD deployment `0x0f62eea9…af4a3d` (block 11916214) and the audit-head anchor `0xf21f8ed7…61ccf7` (block 11939321, entry #15, head `63b7de47…89a3f85`).

```
python scripts/verify_audit.py evidence/audit_sepolia.jsonl                    # OK: 16 entries, head 63b7de47…  (offline)
ACTIVE_NETWORK=sepolia python wallet_engine/scripts/anchor_audit.py --verify   # read-only; needs RPC_URL_SEPOLIA; RESULT: OK
```

Re-running `evidence_run.py --yes` or `anchor_audit.py --yes` sends new Sepolia transactions and spends testnet ETH; do not run them unless you intend to.

7.4 Sepolia wallet activity (complete list)

The throwaway test wallet `0xB89C8A9826cac01E3b4A01873Bd0C27553567609` has sent 15 transactions on Sepolia (testnet, no value) and received the 0.05 test-ETH faucet deposit. The official execution evidence is the 6-send run in `evidence/tx_hashes.csv` (rows 3-8 below). The other transactions are listed here so that the wallet's public history matches this report:

#	Date (UTC)	What it was	Hash
1	14 Oct	Deployment of the TestUSD token	`0x0f62eea9…af4a3d`
2	15 Oct	First verification transfer through plan -> simulate -> execute	`0x1c886a64…1d4599`
3	16 Oct	Evidence run, send 1 (transfer)	`0xd618054f…ee1eef`
4	16 Oct	Evidence run, send 2 (transfer)	`0xc9e0d856…d5db74`
5	16 Oct	Evidence run, send 3 (approve)	`0x852eeff0…08d7d0`
6	16 Oct	Evidence run, send 4 (revoke, an approve of 0)	`0xbc8af874…9c7000`
7	16 Oct	Evidence run, send 5 (transfer)	`0x148f154e…68f0bc`
8	16 Oct	Evidence run, send 6 (transfer)	`0x0ee5931f…60dfc9`
9	17 Oct	Manual test: chat message "send 5 TUSD to Alice" through the protected agent (allowed by the firewall)	`0xcdee3100…432253`
10	17 Oct	Manual test: transfer of 5 TUSD to Alice during a UI session on Sepolia; the sender of this one was not recorded	`0x604db546…861a3f`
11	18 Oct	Audit-log head anchor (self-transaction carrying the head hash, see `evidence/audit_anchor_sepolia.json`)	`0xf21f8ed7…61ccf7`
12	18 Oct	Unintended: transfer of 10 TUSD to Alice	`0x45a1f646…8da617`
13	18 Oct	Unintended: transfer of 30 TUSD to Bob	`0x7aad7b77…02877f`

#	Date (UTC)	What it was	Hash
14	18 Oct	Unintended: approval of 20 TUSD for Alice	`0x913c7574…91e7bc`
15	18 Oct	Unintended: approval of 0 TUSD for Alice (revoke)	`0xf30ad758…60f71a`

Rows 12-15 are the first four scenarios of the evidence script. They were started by an automated test run that was not isolated from the real Sepolia network. This was found while reviewing the wallet history, and every test is now blocked from reaching Sepolia or signing with the real Sepolia key (commit `3d3a990`; after the fix, a full test run left the wallet's transaction count unchanged at 15). These four transactions are not part of the evidence run and are not in `evidence/tx_hashes.csv`. Their net effect is a few test tokens moved to Alice and Bob; the last approval is 0, so no allowance remains open. The lesson is also in the known-risk list (R-19): tests that can reach a real network can spend real (testnet) resources.

---

8. Known-risk list and what is NOT solved

Status: **mitigated** = a control reduces it and is tested · **accepted** = known, left as is for the demo · **not solved** = open. "Owner" is the person who would fix it.

ID	Risk	Severity	Status	What we do about it	Owner
R-01	**NET-01 never fires in normal use.** `chain_id` in `rules.yaml` is `null`, so the "expected" chain is read from the same node that built the plan; the two always agree. NET-01 is proved only by tests that set the id. Wrong-network protection today comes from separate keys per network and from `execute` checking the node's chain.	Medium	not solved (one-line config fix: set `chain_id`)	tests cover NET-01 with an explicit id (`atk-027`, `atk-028`)	Person 2
R-02	**Local state can be reset.** Deleting or truncating `logs/spend_ledger.jsonl` restores the daily cap (verified: 900 spent, file deleted, spent reads 0; dropping the last line also forgets it). The ledger has no stored head, so a cut tail is invisible; editing a line is detected. Velocity memory, open confirmation questions and the used-token list live in process memory and vanish on restart. The kill-switch flag file can be deleted and a restart then unfreezes the wallet.	Medium	accepted	edited lines fail closed; tokens expire in 120 s anyway	Person 2
R-03	**The API has no authentication.** `/api/kill-switch` (including switching it **off**), `/api/confirm`, `/api/execute` and `/api/chat` answer any local caller as the owner. Assumed safe only because uvicorn binds to localhost on a single-user machine.	High if exposed, Low as a local demo	accepted	the documented run command (`uvicorn api.main:app --port 8000`) uses uvicorn's default host 127.0.0.1; do not expose the port	Person 3
R-04	**One process holds everything.** The firewall, the `APPROVAL_SECRET`, the agent and the signing key run together, and keys sit in plain text in `.env`. Code running in that process can mint a valid token (`agent/naive_agent.py` does exactly that, on purpose, to show the cost of having no firewall). The naive agent is guarded by chain id 31337 only; the drainer additionally requires `ACTIVE_NETWORK=local`.	High (by design of a demo)	accepted	`.env` is git-ignored; test keys only; a production design would put the signer behind a separate service or hardware wallet	Person 1
R-05	**Message text goes to a third-party LLM relay.** `LLM_BASE_URL` defaults to a free OpenAI-compatible relay (`api.xkiro.com`). Chat messages, including addresses and amounts, are sent to it. The relay's logging and retention are not known.	Medium	accepted	demo data only; the provider is switchable in `.env`; the firewall does not depend on what the relay returns	Person 3
R-06	**The dry run cannot see hidden balance changes.** `simulate.run` reads the outflow from the same calldata the firewall decodes, so it detects reverts and plan/request mismatches, not a token contract that moves more than its calldata says.	Low	mitigated	TOK-01 allows only our own plain ERC-20	Person 1

ID	Risk	Severity	Status	What we do about it	Owner
R-07	**Look-alike protection only covers the allowlist.** ADR-01 compares against allowlisted addresses; two unfamiliar addresses are never compared. Plan summaries and reasons show shortened addresses (`0x1234…abcd`); the agent's confirmation text shows the full address and the Preview card shows the decoded arguments.	Low	mitigated	ALW-01 asks for any new recipient	Person 2
R-08	**`approvals()` is incomplete on Sepolia.** The free RPC tier refuses wide log searches, so the Wallet card checks only known spenders (allowlist and the local attacker), not every spender.	Low	accepted	local chain scans all `Approval` events; only the block-range refusal triggers the fallback, any other RPC error is raised	Person 1
R-09	**A broadcast that fails is reported as pending.** If `send_raw_transaction` raises after signing, the transaction is reported as pending with its signed hash (so a retry does not pay twice). If the node *rejected* it outright (for example nonce too low) the message still says it "may still be mined": safe but imprecise.	Low	mitigated	`execute.py` computes the hash before sending; `test_execute_pending.py`	Person 1
R-10	**The replay can mislabel a provider outage.** `parse_intent` turns every error (no key, network down, bad reply) into the same refusal error, so the replay page can say "the model produced no action" when the LLM was merely unreachable. The action sent to both agents is still labelled `scripted` in that case.	Low	not solved (wording only)	the `source` field says `scripted` or `llm`	Person 3
R-11	**`anchor_audit.py --verify` and malformed records.** Each record is shape-checked; a bad one is reported `FAILED` with the reason, not a traceback.	Low	mitigated	`test_anchor_audit.py`	Person 1
R-12	**Confused deputy.** Text the owner types or pastes into the chat is `user_chat`, even when it was copied from an attacker. The firewall still applies every limit, the allowlist and the ask rules, so a pasted instruction can at most reach an allowlisted address within limits, or trigger a question.	Medium	accepted	ALW-01, APR-02, LIM-* apply to chat too	Person 2
R-13	**Held, not denied, and approval fatigue.** 4/40 red-team attacks end in *ask*: if a person confirms them, they run. 5/15 legitimate requests also ask, so users may click through. Limits are demo values (auto 50, max 500, daily 1000 TUSD, 5 per 10 minutes) and the allowlist has two entries. There is no sanctions or KYC screening.	Medium	accepted	reasons are written by code from the real transaction; the full address is shown	Person 2
R-14	**Spend is counted when the token is issued.** A transaction that later fails or is refused still consumes daily-cap room; and `execute` consumes the single-use token before its later checks, so a refused execution needs a fresh request. Safe, but repeated failed attempts can use up the daily cap.	Low	accepted	conservative by design	Person 2
R-15	**The red-team result is about the firewall, not the model.** 55/55 and 0/40 do not say how often a real model is fooled. The replay uses the real model's action only if it pays the attacker; otherwise a scripted "fooled model" intent, and the page labels which.	Medium	accepted	stated in every report	Person 2
R-16	**Single-process, single-machine design.** Limits live in memory and local files; a second backend process would not share velocity or open questions. Windows locking is exercised by forcing the code path on Linux, not on a real Windows machine *(not verified on Windows)*. The project was developed and tested on Python 3.14 only.	Low	accepted	`tests/test_audit_locking.py` forces the Windows and portable paths	Person 1
R-17	**Dependencies are mostly unpinned.** `requirements.txt` pins only `web3>=7,<8`; there is no Python lock file and no vulnerability scan is recorded.	Low	not solved	small dependency set	Person 3

ID	Risk	Severity	Status	What we do about it	Owner
R-18	**The audit log is local and only partly anchored.** Only the submitted evidence log is anchored, once, by hand. The live `logs/audit.jsonl` has no stored head, so a cut tail is not detectable there. The anchor transaction itself is not judged by the firewall (a zero-value self-transaction carrying 51 bytes of data; the script refuses any other shape).	Low	accepted	`audit.verify(expected_head=…)` supports an external head	Person 1
R-19	**A test run reached the real Sepolia network (15-18 Oct).** An automated test switched `ACTIVE_NETWORK` to sepolia while `.env` held a funded key, and ran the evidence script: 4 real transactions (7.4, rows 12-15). Now every test has the Sepolia RPC and key replaced by unusable values.	Low	mitigated	`tests/conftest.py` guard (`3d3a990`); a test of the evidence script refuses the rehearsal flag off the local chain	Person 1
R-20	**`agent/naive_agent.py` still shows raw error text in one branch** (the "can't build that transaction" message). It is the deliberately unsafe demo agent and runs on local Anvil only, but the text is not redacted.	Low	accepted	local-only guard (chain id 31337)	Person 3
R-21	**The audit log can show `allow` and then `deny` for one request** when saving the spend record fails after the allow entry was written. The answer given is deny; the first entry misstates it.	Low	accepted	the chain still verifies; the last entry is the final answer	Person 2
R-22	**Stale-lock removal in the portable audit lock can race** (age check and unlink are separate steps). Only the portable path (no `fcntl`, no `msvcrt`) is affected.	Low	accepted	tests force the path on Linux	Person 2

What is NOT solved, in plain words

1. Anyone who can run code on the machine, or call the API from it, can do anything the owner can (R-03, R-04).
2. We have not measured how often a real model proposes a harmful action; we showed only that the firewall holds when it does (R-15).
3. A person who confirms a bad *ask* has authorised it (R-13).
4. Resetting local files weakens the daily cap and the velocity limit (R-02).
5. The "wrong network" rule is not active in the checked-in configuration (R-01).
6. Windows is covered by forced code paths on Linux only (R-16).
7. This is an unaudited demo with test tokens. It is not a compliance system: no KYC, sanctions screening, or regulatory review.